

Voice interface and spoken tool use

Einzelnes druckfertiges Themen-Dossier.

Erstellt 2026-06-17

Warum ich es gebaut habe

Ich wollte mit meinem System sprechen können, aber Sprache ist für Tools riskant: sie ist schnell, ungenau und kann trotzdem Aktionen auslösen. Deshalb brauchte ich Muting, Tool-Grenzen, Fallbacks und Schutz gegen gefährliche Befehle.

Architektur und Evidenz

Grenze: Terminal/Prototyp und Planungsstand sauber trennen; Wake-Word/macOS-Teile nur behaupten, wenn frisch getestet.

- Python streamt Mikrofon-Audio in Gemini Live.
- Tools erlauben Vault Search/Read/Write, Deep Thinking, Brain Status und begrenzte Shell.
- Gefährliche Shell-Muster werden blockiert, Ausführung bekommt Timeouts.
- Spätere Planung umfasst Wake Word, lokale STT, Hybrid-TTS, Barge-in und Privacy-Grenzen.

Was ich zeigen kann

- Ich habe Voice als tool-fähige Kontrolloberfläche gebaut, nicht nur als Chat-Spielerei.
- Ich habe Datenschutz, Bestätigung und Unterbrechbarkeit von Anfang an mitgedacht.
- Ich habe hochwertige Stimmen mit praktischer Tool-Nutzung verbunden.
- Voice senkt Reibung beim Auslösen und muss deshalb Reibung beim Risiko erhöhen.
- Barge-in ist ein Sicherheitsfeature, weil ich das System sofort stoppen können muss.
- Eine schöne Stimme ist wertlos, wenn Autorität und Privacy unklar bleiben.
- Öffentliche Demo ohne private Mikrofon-/Vault-Daten bauen.
- Safe Tools und confirmation-required Tools klar trennen.
- Transkript-Audit und Korrekturpfade ergänzen.